

ISTQB® CTFL v4.0

Chapter 4

Test Analysis and Design



Chapter 4 learning route

4

4.1

Technique overview

4.2

Black-box techniques

4.3

White-box techniques

4.4

Experience-based techniques

4.5

Collaboration-based approaches

4.1 Test Techniques Overview

- FL-4.1.1 (K2) Distinguish black-box, white-box and experience-based test techniques

4.2 Black-box Test Techniques

- FL-4.2.1 (K3) Use equivalence partitioning to derive test cases
- FL-4.2.2 (K3) Use boundary value analysis to derive test cases
- FL-4.2.3 (K3) Use decision table testing to derive test cases
- FL-4.2.4 (K3) Use state transition testing to derive test cases

4.3 White-box Test Techniques

- FL-4.3.1 (K2) Explain statement testing
- FL-4.3.2 (K2) Explain branch testing
- FL-4.3.3 (K2) Explain the value of white-box testing

4.4 Experience-based Test Techniques

- FL-4.4.1 (K2) Explain error guessing
- FL-4.4.2 (K2) Explain exploratory testing
- FL-4.4.3 (K2) Explain checklist-based testing

4.5. Collaboration-based Test Approaches

- FL-4.5.1 (K2) Explain how to write user stories in collaboration with developers and business representatives
- FL-4.5.2 (K2) Classify the different options for writing acceptance criteria
- FL-4.5.3 (K3) Use acceptance test-driven development (ATDD) to derive test cases



INTERACTIVE: Ask learners which technique they already know from practice.

Learning objectives mapped to syllabus



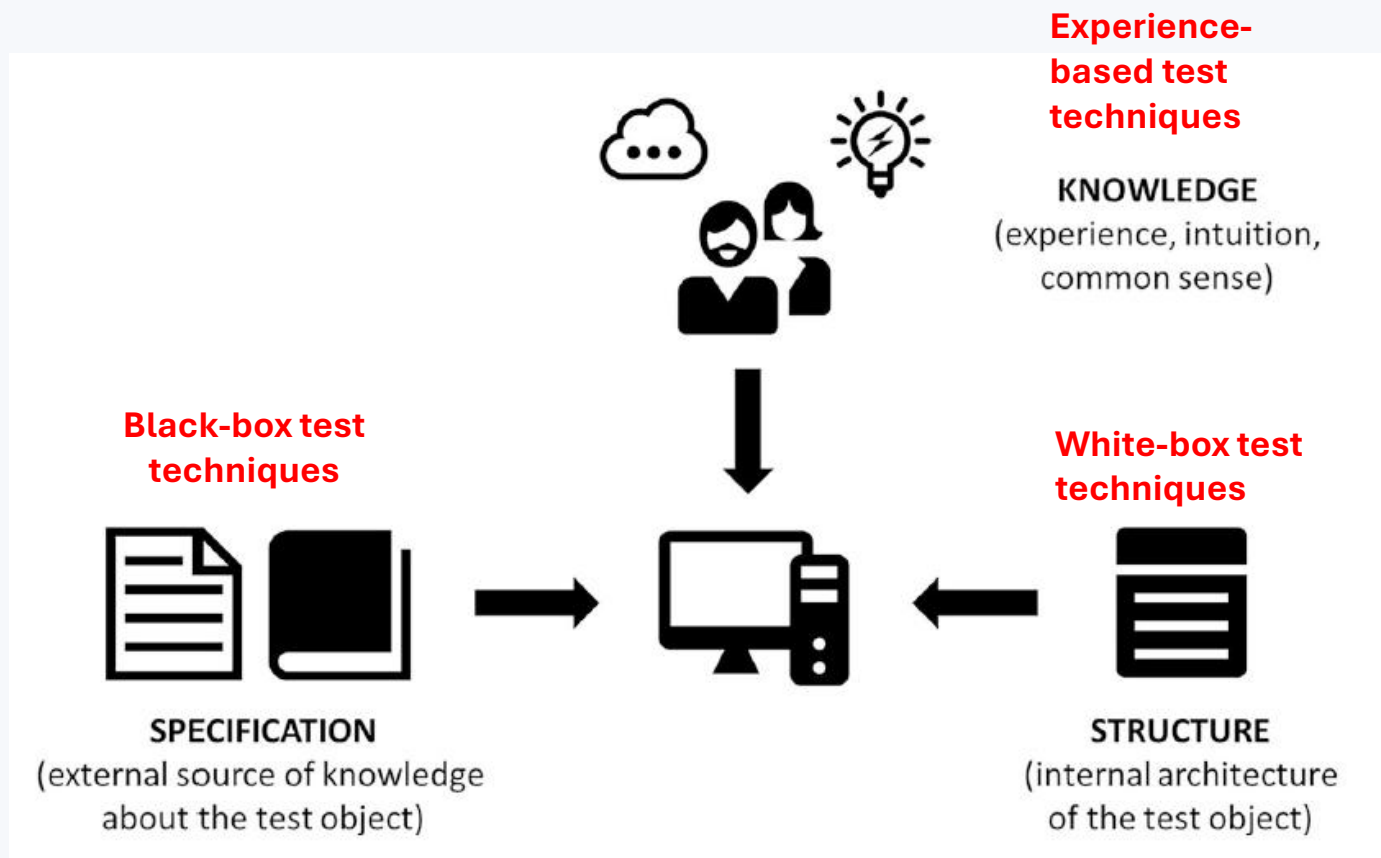
Section	K-level	Learning focus
4.1.1	K2	Distinguish black-box, white-box and experience-based techniques
4.2.1–4.2.4	K3	Use EP, BVA, decision tables and state transitions
4.3.1–4.3.3	K2	Explain statement/branch testing and white-box value
4.4.1–4.4.3	K2	Explain error guessing, exploratory and checklist-based testing
4.5.1–4.5.3	K2/K3	User stories, acceptance criteria and ATDD

INTERACTIVE Quick check: identify which Chapter 4 objectives require actual calculation or test-case derivation.

4.1 Test techniques overview



Techniques support systematic test analysis and design



Why techniques help

- Develop a small but sufficient test set
- Define test conditions and coverage items
- Derive test data and expected results
- Reduce chance of missing important cases



Three knowledge sources

- Specification → black-box
- Structure → white-box
- Experience, intuition and common sense → experience-based
- In practice, combine them

INTERACTIVE Ask: why do we need test techniques if the requirements already describe the system?

4.1.1 Black-box, white-box and experience-based



The three technique families complement each other

Black-box

- Black-box techniques are also called behavioral or behavior-based. They can be used for both functional and non-functional testing and concentrate on externally visible behavior.
- Derived from requirements, user stories, use cases, business processes; focuses on inputs and outputs.

White-box

- White-box techniques are structure-based. They are often associated with component testing, but can also be used at integration, system and acceptance levels when a structure is available.
- Derived from code, architecture, detailed design or internal flow; focuses on structure and coverage.

Experience-based

- Experience-based techniques are valuable when formal documentation is incomplete or when the tester has domain insight.
- Derived from knowledge of testers, developers, users and stakeholders; useful with weak specifications.

Classification examples

- Boundary value analysis → black-box
- Decision table testing → black-box
- Branch testing → white-box
- Exploratory testing → experience-based

INTERACTIVE Classify these: equivalence partitioning, branch testing, exploratory testing, checklist-based testing.



4.2 Black-box Test Techniques

4.2.1 Equivalence Partitioning (EP)

Divide values into groups expected to be processed the same way

Equivalence partitioning divides data into partitions (also known as equivalence classes) in such a way that all the members of a given partition are expected to be processed in the same way.

There are equivalence partitions for both valid and invalid values.

- **Valid values** are values that should be accepted by the component or system. An equivalence partition containing valid values is called a “valid equivalence partition.”
- **Invalid values** are values that should be rejected by the component or system. An equivalence partition containing invalid values is called an “invalid equivalence partition.”
- **Partitions can be identified for any data element related to the test object**, including inputs, outputs, internal values, time-related values (e.g., before or after an event) and for **interface parameters** (e.g., integrated components being tested during integration testing).
- Any partition may be divided into sub partitions if required.
- **Each value must belong to one and only one equivalence partition.**
- When invalid equivalence partitions are used in test cases, they should be tested individually, i.e., not combined with other invalid equivalence partitions, to ensure that failures are not masked. Failures can be masked when several failures occur at the same time but only one is visible, causing the other failures to be undetected.

To achieve **100% coverage with this technique**, **test cases must cover all identified partitions** (including invalid partitions) by using a minimum of one value from each partition. Coverage is measured as the **number of equivalence partitions tested by at least one value, divided by the total number of identified equivalence partitions**, normally expressed as a percentage. Equivalence partitioning is applicable at all test levels.

INTERACTIVE Create partitions for age input: valid 18–65, invalid below 18 and above 65.

4.2 Black-box Test Techniques



4.2.1 Equivalence Partitioning (EP)

Divide values into groups expected to be processed the same way



Core EP rules

- Partitions are non-empty and non-overlapping
- Every relevant value belongs to one and only one partition
- Valid partitions contain values that should be accepted
- Invalid partitions contain values that should be rejected



Coverage

- Coverage items = equivalence partitions
- 100% EP coverage exercises every identified partition at least once
- For multiple partition sets, Each Choice coverage covers every partition from every set
- Invalid partitions are normally tested individually to avoid masking

EP reduces many possible values into representative groups. The key assumption is that values in the same partition are processed in the same way, so one representative may reveal the same defect as other values in that partition.

Do not combine several invalid partitions in one test case when the objective is to check invalid handling. If multiple invalid values are used together, one failure can hide another, producing defect masking.

INTERACTIVE Create partitions for age input: valid 18–65, invalid below 18 and above 65.

4.2.1 EP examples



Identify valid and invalid classes before selecting test values

Specification	Valid Classes	Invalid Classes
Integer range 1..10	Between 1 to 10	<1; >10; non-integer
Not less than 2 and not more than 10 lower letters	2 to 10 chars	<2 chars; >10 chars
'A', 'B', 'C', 'D'	A; B; C; D;	others



How to choose representative values

- Pick at least one value from each partition
- For EP only, the value can be any member of the partition
- For BVA, values near edges become more important
- Document why each value represents its partition

INTERACTIVE Pair activity: add one more invalid partition for each row and explain whether it should be tested separately.

4.2.1 EP worked example: age discount



Accepted age/no-discount range: 18–65 inclusive



Example of EP representatives: 17, 42, 66

This example separates EP from BVA. With pure EP, one value from each partition can be enough: one below 18, one inside 18–65, and one above 65.

INTERACTIVE Ask learners which values are enough for EP and which extra values are caused by BVA thinking.

4.2.2 Boundary Value Analysis (BVA)



Defects often occur at the edges of ordered partitions



When BVA applies

- Boundary value analysis (BVA) is an **extension of equivalence partitioning**, but can only be used when the partition is ordered, consisting of numeric or sequential data. The minimum and maximum values (or first and last values) of a partition are its boundary values.
- **Behavior at the boundaries of equivalence partitions is more likely to be incorrect than behavior within the partitions.** It is important to remember that both specified and implemented boundaries may be displaced to positions above or below their intended positions, may be omitted altogether, or may be supplemented with unwanted additional boundaries. Boundary value analysis and testing will reveal almost all such defects by forcing the software to show behaviors from a partition other than the one to which the boundary value should belong.
- Boundary value analysis can be applied at all test levels. This technique is generally used to test requirements that call for **a range of numbers** (including dates and times). Boundary coverage for a partition is measured as the number of boundary values tested, divided by the total number of identified boundary test values, normally expressed as a percentage.



Coverage idea

- Coverage items = boundary test values
- Coverage = tested boundary values / identified boundary test values
- 2-value BVA: boundary + nearest neighbor in adjacent partition
- 3-value BVA: boundary + both neighbors

INTERACTIVE For range 18–65, list the 2-value BVA and 3-value BVA sets.

4.2.2 Boundary Value Analysis (BVA)



Defects often occur at the edges of ordered partitions



BVA variants

There are two main variants of the BVA—the so-called **2-value BVA** and **3-value BVA**.

2-Value BVA

- In the 2-value BVA, for each identified boundary value, that value and its nearest neighbor not belonging to the partition to which the boundary value belongs are selected for testing.
- For example, if we consider boundaries of the partition $P = \{1, 2, 3, 4, 5, 6\}$ in the integer domain (i.e., the values 1 and 6), hence we select for testing:
 - ☐ Value of 1 (as a boundary value of P)
 - ☐ Value of 0 (as the nearest neighbor of 1 not belonging to P)
 - ☐ Value of 6 (as a boundary value of P)
 - ☐ Value of 7 (as the nearest neighbor of 6 not belonging to P)

INTERACTIVE For range 18–65, list the 2-value BVA and 3-value BVA sets.

4.2.2 Boundary Value Analysis (BVA)



Defects often occur at the edges of ordered partitions



BVA variants

3-Value BVA

- In the 3-value BVA for each identified boundary value, we take that value and both of its neighbors for testing, regardless of which partitions they belong to.
- In the example before this, we would select for testing:
 - ☐ Value of 1 (as a boundary value of P)
 - ☐ Value of 0 (as the left neighbor of 1)
 - ☐ Value of 2 (as the right neighbor of 1)
 - ☐ Value of 6 (as a boundary value of P)
 - ☐ Value of 5 (as the left neighbor of 6)
 - ☐ Value of 7 (as the right neighbor of 6)

INTERACTIVE For range 18–65, list the 2-value BVA and 3-value BVA sets.

4.2.2 2-value vs 3-value BVA



Use unique values and avoid duplicate counting

2-BVA

Specification	Tests	No. of tests cases
Integer range 1..10	0/1, 10/11	4
Not less than 2 and not more than 10 lower letters	1/2 chars, 10/11 chars	4
'A', 'B', 'C', 'D'	Pred('A')/A, D/succ('D')	4

3-BVA

Specification	Tests	No. of tests cases
Integer range 1..10	0/1/2, 9/10/11	6
Not less than 2 and not more than 10 lower letters	1/2/3 chars, 9/10/11 chars	6
'A', 'B', 'C', 'D'	Pred('A')/A/B, C/D/succ('D')	6



Exam tip

- Check whether the question asks for 2-value or 3-value
- Check whether invalid values are feasible
- Count unique values only
- Match values to lower and upper boundaries

INTERACTIVE Calculate the total unique values for 2-value and 3-value BVA for 1–100.

4.2.2 BVA age-discount exercise



Children under 18, no discount 18–65, senior over 65

The system assigns a ticket discount to passengers under 18 (children discount) and over 65 (senior discount). A passenger between the ages of 18 and 65 is not entitled to a discount. We want to check the correctness of the allocation of the discount.

1. Determine the EP.
2. Determine the 2-BVA and 3-BVA.

Partitions

0–17 children
18–65 no discount
66+ senior

2-value BVA

0, 17, 18, 65, 66
(if -1 infeasible)

3-value BVA

0,1,16,17,18,19,64,65,66,67



Why 3-value adds more

- It checks just below, at and just above each boundary
- It may reveal misplaced inequalities such as $<$ vs $\leq$
- It is useful when the component is high-risk
- It increases test cases, so use risk-based judgment

INTERACTIVE Before revealing the 3-value set, ask learners to derive it on paper.

4.2.3 Decision Table Testing



Use decision tables for combinations of conditions and actions

- The techniques of EP and BVA are often applied to **specific situations or inputs**. However, if different combinations of inputs result in different actions being taken, this can be more difficult to show using EP and BVA, which tend to be more focused on the user interface.
- The other two specification-based techniques, decision table testing and state transition testing, are more focused on **business logic or business rules**.

IF (condition) THEN (action)

Examples:

- **IF** (customerAge < 18) **THEN** (assign a ticket discount);
- **IF** (monthlySalary > 10000) **THEN** (grant bank loan **AND** offer a gold card).

Decision table testing is stronger than checking individual inputs when the behavior depends on combinations. It helps reveal contradictions, missing rules and impossible combinations.

INTERACTIVE Identify conditions and actions for a “free shipping” rule.

4.2.3 Decision Table Testing



Use decision tables for combinations of conditions and actions



Core idea

- Rows contain conditions and resulting actions
- Columns contain decision rules
- Excellent for business rules and complex logic
- Covers combinations that EP/BVA may overlook



Notation and coverage

- Y/T/1 = condition true
- N/F/0 = condition false
- – means value does not matter
- X means action occurs
- 100% coverage exercises every feasible rule

- A full decision table has enough columns (test cases) to cover every combination of conditions. By deleting columns that do not affect the outcome, the number of test cases can decrease considerably. For example, by removing impossible combinations of conditions.
- The common minimum coverage standard for decision table testing is to have at least one test case per decision rule in the table. This typically involves covering all combinations of conditions. Coverage is measured as the number of decision rules tested by at least one test case, divided by the total number of decision rules, normally expressed as a percentage.
- The strength of decision table testing is that it helps to identify all the important combinations of conditions, some of which might otherwise be overlooked. It also helps in finding any gaps in the requirements. It may be applied to all situations in which the behavior of the software depends on a combination of conditions, at any test level.

INTERACTIVE Identify conditions and actions for a “free shipping” rule.

4.2.3 Worked example: loan inputs



Two conditions produce four combinations

Take a scenario of a loan application, where you can **enter** the amount of the monthly repayment or the number of years you want to take to pay it back (the term of the loan).

- The two conditions are the **repayment amount** and the **term**.
- Now, identify all of the combinations of **True** and **False**.

TABLE 4.4 Decision table with combinations and outcomes

Conditions	Rule 1	Rule 2	Rule 3	Rule 4
<i>Repayment amount has been entered</i>	T	T	F	F
<i>Term of loan has been entered</i>	T	F	T	F
Actions/Outcomes				
<i>Process loan amount</i>	Y	Y		
<i>Process term</i>	Y		Y	

Any omitted condition / outcome?

Error message

Y

INTERACTIVE Ask learners to identify whether Rule 4 should show an error message.

4.2.4 State Transition Testing



Model states, events, transitions and actions

- A state transition diagram shows the possible software states, as well as how the software enters, exits, and transitions between states.
- A transition is initiated by an event (e.g., user input of a value into a field). The event results in a transition. The same event can result in two or more different transitions from the same state. The state change may result in the software taking an action (e.g., outputting a calculation or error message).
- A state transition table shows all valid transitions and potentially invalid transitions between states, as well as the events, and resulting actions for valid transitions.
- State transition diagrams normally show only the valid transitions and exclude the invalid transitions.
- Tests can be designed to cover a typical sequence of states, to exercise all states, to exercise every transition, to exercise specific sequences of transitions, or to test invalid transitions.
- State transition testing is used for menu-based applications and is widely used within the embedded software industry. The technique is also suitable for modeling a business scenario having specific states or for testing screen navigation. The concept of a state is abstract – it may represent a few lines of code or an entire business process.
- Coverage is measured as the number of valid and invalid transitions exercised or attempted to be covered by executed test cases, divided by the total number of valid and invalid transitions, and is expressed as a percentage

INTERACTIVE Ask learners: where do you see state behavior in a mobile app?

4.2.4 State Transition Testing



Model states, events, transitions and actions



What the model shows

- State = condition or situation of the system
- Event = trigger that causes a transition
- Transition = movement from one state to another
- Action = output or behavior caused by the transition



Coverage options

- All states coverage
- Valid transitions coverage / 0-switch coverage
- All transitions coverage including invalid transitions
- Specific transition sequences

INTERACTIVE Ask learners: where do you see state behavior in a mobile app?

4.2.4 State transition example: PIN entry



A sequence of invalid PINs changes system state

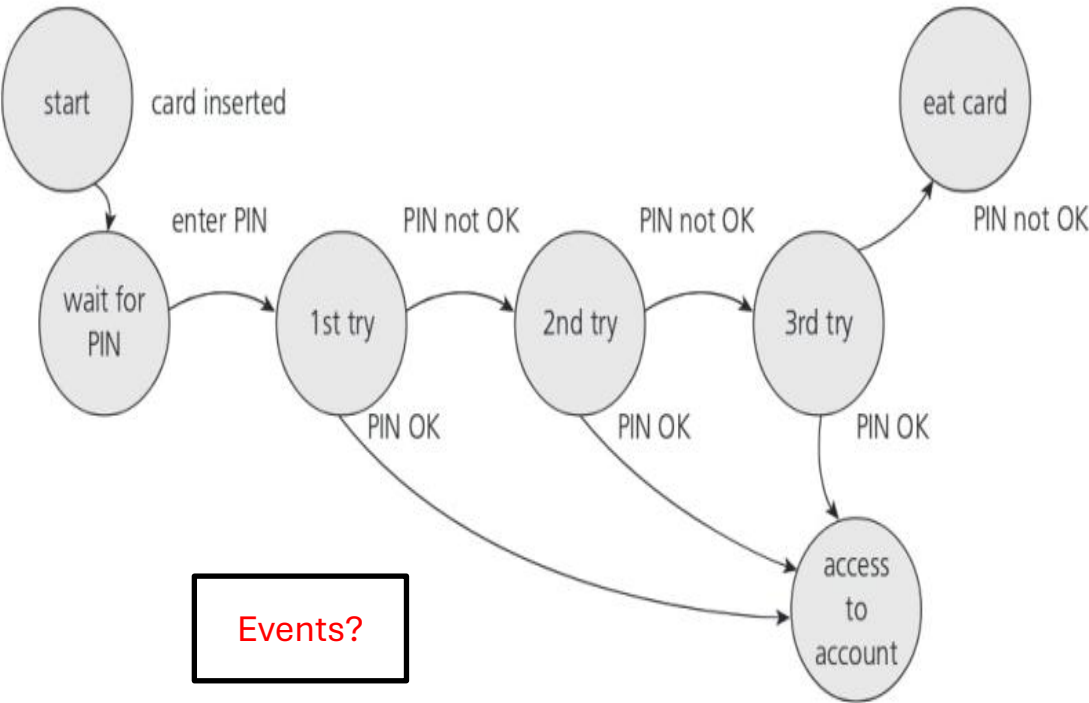


TABLE 4.10 State table for the PIN example

	Insert card	Valid PIN	Invalid PIN
S1) Start state	S2	–	–
S2) Wait for PIN	–	S6	S3
S3) 1st try invalid	–	S6	S4
S4) 2nd try invalid	–	S6	S5
S5) 3rd try invalid	–	–	S7
S6) Access account	–	?	?
S7) Eat card	S1 (for new card)	–	–

FIGURE 4.2 State diagram for PIN entry

Coverage hierarchy: valid transition coverage normally guarantees all states coverage, but all states coverage does not guarantee all transitions. All transitions coverage includes attempts to exercise invalid transitions and is stronger.

INTERACTIVE Give an event sequence that reaches “eat card”. Then identify the transitions covered.

4.2.4 Coverage strength in state transition testing



Coverage criteria are not equally strong

All states

Visit every state at least once
Weakest criterion

Valid transitions

Exercise every valid single transition
Commonly used

All transitions

Exercise valid + attempt invalid transitions
Stronger; important for critical systems

Valid transitions coverage is sometimes called 0-switch coverage. It focuses on single valid transitions.

All transitions coverage requires both valid transitions and attempted invalid transitions. For safety-critical or mission-critical software, stronger transition coverage may be expected.



Common exam trap

- Visiting all states does not mean all arrows were used
- One sequence can cover many transitions
- Invalid transitions should often be tested individually
- Coverage denominator must match the chosen criterion (eg: Chosen criterion: All states coverage , Coverage denominator: Total number of states etc.)

INTERACTIVE Which criterion is stronger: all states or valid transitions? Explain using the PIN example.

4.3 White-box Test Techniques



White-box testing is based on the internal structure of the test object

Possible structures

White-box testing is based on the internal structure of the test object. Most often, white-box testing is associated with **component testing**, in which the model of the test object is the **internal structure of the code**, represented, for example, in the form of a so-called control flow graph (CFG).

However, it is important to note that white- box test techniques can be applied to all test levels, e.g.:

- At component testing (example structure: CFG)
- At integration testing (example structures: call graph, API)
- At system testing (example structures: business process modeled in BPMN language,3 program menu)
- At acceptance testing (example structure: website pages structure)



Chapter 4 focus

- Statement testing
- Branch testing
- Value of structural coverage
- Complementing black-box tests

INTERACTIVE Ask learners to name a structure other than code that could be used for white-box testing.

4.3.1 Statement testing and coverage



Exercise executable statements in the code



Statement coverage formula

- Statement coverage = executed statements / total executable statements × 100%
- Comments are not executable statements
- ELSE alone is usually syntax, not an executable instruction
- Simplest and weakest white-box coverage

Example of a simple code snippet:

```
1.  INPUT x, y // two natural
    numbers
2.  IF (x > y) THEN
3.    z := x - y
    ELSE
4.    z := y - x
5.    IF (x > 1) THEN
6.      z := z * 2
7.  RETURN z
```

Executable statements: 1–7

- Executable statements are marked with the numbers 1–7 in this code.
- The text starting with the “//” characters is a comment and is not executed when the program runs.
- In general, the code is executed sequentially, line by line, unless some jumps in the control flow occur (e.g., in the decision statements 2 and 5 in our code).
- The keyword ELSE is only part of the syntax of the IF-THEN-ELSE statement and by itself is not treated as executable instruction (because there is nothing to execute here).

INTERACTIVE For one test input, mark which numbered statements are executed.

4.3.1 Statement coverage worked example



Coverage is a measurement, not a guarantee of quality

Test	Input	Executed statements	Coverage
T1	x=5, y=3	1,2,3,5,6,7	6/7 = 86%
T2	x=2, y=4	1,2,4,5,6,7	6/7 = 86%
T1+T2	both	1,2,3,4,5,6,7	7/7 = 100%



Important

- 100% statement coverage does not imply 100% branch coverage
- Coverage only tells which items were exercised
- It says nothing by itself about correctness of assertions
- Use coverage to identify gaps and guide further testing

INTERACTIVE Can 40% coverage plus 65% coverage mean 105%? Why not?

4.3.2 Branch testing and coverage



Exercise decision outcomes and control-flow branches



Branch coverage formula

- Branch coverage = branches exercised / total branches × 100%
- Branches are outcomes of decisions such as IF and CASE
- Full branch coverage generally implies full statement coverage
- But not vice versa



Why it is stronger

- Forces true and false outcomes to be checked
- Reveals untested decision logic
- Highlights paths hidden by statement coverage
- Useful for risk-based structural testing

- ❑ A branch is a **transfer of control between two nodes in the control flow graph**, which shows the possible sequences in which source code statements are executed in the test object. Each transfer of control can be either unconditional (i.e., straight-line code) or conditional (i.e., a decision outcome).
- ❑ In branch testing the coverage items are **branches** and the aim is to design test cases to exercise branches in the code until an acceptable level of coverage is achieved.
- ❑ Coverage is measured as the number of branches exercised by the test cases divided by the total number of branches, and is expressed as a percentage.
- ❑ When 100% branch coverage is achieved, all branches in the code, unconditional and conditional, are exercised by test cases.

Conditional branches typically correspond to a **true or false** outcome from an “if...then” decision, an outcome from a switch/case statement, or a decision to exit or continue in a loop. However, exercising a branch with a test case will not detect defects in all cases. For example, it may not detect defects requiring the execution of a specific path in a code.

INTERACTIVE For the code example, count the branches from IF x>y and IF x>1.

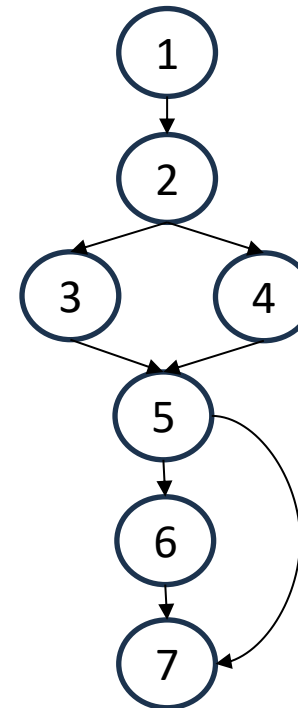
4.3.2 Branch testing and coverage



Exercise decision outcomes and control-flow branches

Example of a simple code snippet:

1. **INPUT** x, y // two natural numbers
2. **IF** $(x > y)$ **THEN**
3. $z := x - y$
- ELSE**
4. $z := y - x$
5. **IF** $(x > 1)$ **THEN**
6. $z := z * 2$
7. **RETURN** z



INTERACTIVE For the code example, count the branches from IF $x > y$ and IF $x > 1$.

4.3.3 Value of white-box testing



Structural coverage reveals gaps that black-box tests may miss

- When 100% statement coverage is achieved, it ensures that all executable statements in the code have been tested at least once, but it does not ensure that all decision logic has been tested. Of the two white-box techniques discussed in this syllabus, statement testing may provide less coverage than branch testing.
- When 100% branch coverage is achieved, it executes all decision outcomes, which includes testing the true outcome and also the false outcome, even when there is no explicit false statement (e.g., in the case of an IF statement without an else in the code).
- Branch coverage includes statement coverage. This means that any set of test cases achieving 100% branch coverage also achieves 100% statement coverage (but not vice versa).



Value

- Measures how thoroughly structure has been exercised
- Finds gaps in black-box test suites
- Can guide additional tests for code, architecture or workflows
- Supports objective coverage reporting



Limits

- Coverage does not guarantee correctness
- May not test missing requirements
- Can miss data-related and business-rule defects
- Needs access to structural information

INTERACTIVE Ask: what can 100% branch coverage still miss?

4.4 Experience-based Test Techniques



Use human knowledge where formal sources are insufficient

- When applying experience-based test techniques, the test cases are derived from the **tester's skill and intuition, and their experience** with similar applications and technologies.
- These techniques can be helpful in identifying tests that were not easily identified by other more systematic techniques.
- Depending on the tester's approach and experience, **these techniques may achieve widely varying degrees of coverage and effectiveness**. Coverage can be difficult to assess and may not be measurable with these techniques.

Error guessing

Predict likely defects based on experience

Exploratory testing

Design, execute and learn simultaneously

Checklist-based testing

Use checklist items to guide testing



Common features

- Depends heavily on tester skill and knowledge
- Useful when documentation is weak or time is short
- Complements formal black-box and white-box techniques
- Can reveal practical usability and workflow issues

INTERACTIVE Ask learners to share one “usual suspect” defect they often look for.

4.4.1 Error guessing



Use knowledge of likely defects and failures

Error guessing is a technique used **to anticipate the occurrence of errors, defects, and failures, based on the tester's knowledge**, including:

- How the application has worked in the past
- What kind of errors tend to be made
- Failures that have occurred in other applications

Fault attacks are a methodical approach to the implementation of error guessing. This technique requires the tester:

- To create or acquire a list of possible errors, defects and failures, and
- To design tests that will identify defects associated with the errors, expose the defects, or cause the failures.
- These lists can be built based on experience, defect and failure data, or from common knowledge about why software fails.



Sources for guesses

- Past defects and production incidents
- Tester and developer experience
- Typical implementation mistakes
- Known risky areas and technologies
- Checklists and defect taxonomies



Examples

- Off-by-one boundaries
- Null / blank / special characters
- Invalid combinations
- Timeouts and interrupted workflows
- Permissions and role-based access

INTERACTIVE For a login screen, brainstorm five error-guessing tests in two minutes.

4.4.2 Exploratory testing



Learning, test design and execution happen at the same time

- In exploratory testing, **informal** (not pre-defined) tests are designed, executed, logged, and evaluated dynamically during test execution. The test results are used to learn more about the component or system, and to create tests for the areas that may need more testing.
- Exploratory testing is sometimes conducted using **session-based testing** to structure the activity. In session-based testing, exploratory testing is conducted within a defined time-box, and the tester uses a test charter containing test objectives to guide the testing. The tester may use test session sheets to document the steps followed and the discoveries made.
- Exploratory testing is **most useful when there are few or inadequate specifications or significant time pressure on testing**. Exploratory testing is also useful to complement other more formal testing techniques.
- Exploratory testing is strongly associated with reactive test strategies (see section 5.2.2).
- Exploratory testing can incorporate the use of other black-box, white-box, and experience-based techniques.



Good fit when

- Specifications are weak or changing
- Time is limited but skilled testers are available
- A new or unfamiliar feature needs rapid learning
- You want to complement scripted testing



How to control it

- Use a test charter
- Time-box the session
- Record notes, defects and coverage observations
- Debrief and decide follow-up tests

INTERACTIVE Create a one-sentence charter for exploring an online order-history feature.

4.4.3 Checklist-based testing



Use checklist items to guide testing while allowing tester judgment

In checklist-based testing, testers:

- **design, implement, and execute tests to cover test conditions found in a checklist.**
- As part of analysis, testers create a new checklist or expand an existing checklist, but testers may also use an existing checklist without modification. Such **checklists can be built based on experience, knowledge** about what is important for the user, or an understanding of why and how software fails.
- Checklists can be created to support various test types, **including functional and non-functional testing**. In the absence of detailed test cases, checklist-based testing can provide guidelines and a degree of consistency. As these are high-level lists, some variability in the actual testing is likely to occur, resulting in potentially greater coverage but less repeatability.



Checklist characteristics

- Can be generic or product-specific
- May cover requirements, usability, security or common defects
- Does not prescribe exact step-by-step test cases
- Needs maintenance to stay useful



Good checklist item examples

- Error messages are clear and helpful
- Mandatory fields are validated
- Data is saved only after confirmation
- Role permissions are enforced
- Navigation works after interruption

INTERACTIVE Rewrite “the program works correctly” into three testable checklist items.

4.5 Collaboration-based Test Approaches



Use shared examples to clarify expectations before development



Main approaches

- Collaborative user story writing
- Acceptance criteria definition
- Acceptance Test-Driven Development (ATDD)
- Examples and conversations with business representatives



Why collaboration helps

- Builds shared understanding
- Reveals ambiguity early
- Connects tests to business value
- Creates acceptance tests before implementation

INTERACTIVE Ask learners: who should participate in writing acceptance criteria and why?

4.5.1 User stories in collaboration



A user story is a promise for conversation, not a full specification by itself

A **user story** represents a feature that will be valuable to either a user or purchaser of a system or software. User stories have three critical aspects (Jeffries 2000), called together the “3C’s”:

- Card – the medium describing a user story (e.g., an index card, an entry in an electronic board)
- Conversation – explains how the software will be used (can be documented or verbal)
- Confirmation – the acceptance criteria (acceptable conditions)



Common format

- As a [role]
- I want [capability]
- So that [business value]
- Discuss examples and constraints collaboratively

The most common format for a user story is “As a [role], I want [goal to be accomplished], so that I can [resulting business value for the role]”, followed by the acceptance criteria.



INVEST reminders

- Independent
- Negotiable
- Valuable
- Estimable
- Small
- Testable

Good user stories should be: **Independent, Negotiable, Valuable, Estimable, Small and Testable (INVEST)**. If a stakeholder does not know how to test a user story, this may indicate that the user story is not clear enough, or that it does not reflect something valuable to them, or that the stakeholder just needs help in testing (Wake 2003).

- **Collaborative authorship** of the user story can use techniques such as brainstorming and mind mapping. The collaboration allows the team to obtain a shared vision of what should be delivered, by taking into account three perspectives: business, development and testing.

INTERACTIVE Improve this story: “As a user, I want reports so that I can see data.”

4.5.2 Acceptance criteria options



Acceptance criteria define when a story is acceptable

- Acceptance criteria for a user story are the **conditions that an implementation of the user story must meet to be accepted by stakeholders**. From this perspective, acceptance criteria may be viewed as the test conditions that should be exercised by the tests.
- Acceptance criteria are usually a result of the Conversation (see section 4.5.1).
- Acceptance criteria are used to:
 - Define the **scope** of the user story
 - Reach **consensus** among the stakeholders
 - Describe both **positive and negative** scenarios
 - Serve as a **basis** for the user story acceptance testing (see section 4.5.3)
 - Allow **accurate** planning and estimation

INTERACTIVE Classify acceptance criteria as scenario-oriented or rule-oriented.

4.5.2 Acceptance criteria options



Acceptance criteria define when a story is acceptable

There are several ways to write acceptance criteria for a user story. The two most common formats are:

Scenario-oriented

- Given–When–Then
- Best for behavior examples
- (e.g., Given/When/Then format used in Behavior-Driven Development (BDD), see section 2.1.3)

Rule-oriented

- Bullet rules or business constraints
- Best for simple rules
- (e.g., bullet point verification list, or tabulated form of input-output mapping)

Custom format

- Most acceptance criteria can be documented in one of these two formats. However, the team may use another, custom format, as long as the acceptance criteria are well-defined and unambiguous.
- Tables, examples or models
- Use what stakeholders understand



Good criteria are

- Testable and unambiguous
- Connected to user value
- Clear about valid, invalid and exception behavior
- Understandable by business and technical people

INTERACTIVE Classify acceptance criteria as scenario-oriented or rule-oriented.

4.5.3 Acceptance Test-Driven Development (ATDD)



Derive acceptance tests before implementation

- ATDD is a test-first approach (see section 2.1.3). Test cases are created prior to implementing the user story. The test cases are created by **team members with different perspectives, e.g., customers, developers, and testers** (Adzic 2009). Test cases may be executed manually or automated.

First step

- A **specification workshop** where the user story and (if not yet defined) its acceptance criteria are analyzed, discussed, and written by the team members.
- Incompleteness, ambiguities, or defects in the user story are resolved during this process.

Next step

- **To create the test cases.** This can be done by the team as a whole or by the tester individually.
- The test cases are based on the acceptance criteria and can be seen as examples of how the software works. This will help the team implement the user story correctly.

- Typically, the **first test cases are positive**, confirming the correct behavior without exceptions or error conditions, and comprising the sequence of activities executed if everything goes as expected. After the positive test cases are done, the team should **perform negative testing**. Finally, the team should cover **non-functional quality characteristics** as well (e.g., performance efficiency, usability).
- Test cases should be expressed in a way that is understandable for the stakeholders. Typically, test cases contain **sentences in natural language** involving the necessary preconditions (if any), the inputs, and the postconditions.
- The test cases **must cover all the characteristics of the user story** and should not go beyond the story. However, the acceptance criteria may detail some of the issues described in the user story. In addition, no two test cases should describe the same characteristics of the user story.
- Since examples and tests are the same, these terms are often used interchangeably. During the test design the test techniques described in sections 4.2, 4.3 and 4.4 may be applied.

INTERACTIVE From the order-history user story, write one Given–When–Then acceptance test.

4.5.3 Acceptance Test-Driven Development (ATDD)



Derive acceptance tests before implementation

Example: Suppose a team intends to test the following user story.

User story US-002-02

As a logged-in bank customer

I want to be able to transfer money to another account

So that be able to transfer funds between accounts

Possible Acceptance criteria:

- (AC1) the amount of the transfer cannot exceed the account balance
- (AC2) target account must be correct
- (AC3) for valid data (amount, account numbers) the source account balance decreases and the destination account balance increases by the amount of the transfer
- (AC4) the transfer amount must be positive and represent the correct amount of money (i.e., be accurate to two decimal places max)

Possible positive tests:

- TC1: make a “typical” transfer, to another correct account; with a correct expected result.
- TC2: make a correct amount of transfer of the entire account balance to another correct
- Account

Possible negative tests:

- TC3: attempt to make a transfer to another, correct account, when the transfer amount is greater than the balance of the source account (verification of AC1)
- TC4: attempt to make a transfer with correct transfer amount and balance, but to a non-existent account (verification of AC2)

INTERACTIVE From the order-history user story, write one Given–When–Then acceptance test.

Chapter 4 mini-quiz



Use as formative assessment before sample exam questions

1. Which technique uses specification and ignores internal structure?

2. What is the difference between EP and BVA?

3. What is the denominator for decision table coverage?

4. Why is branch coverage stronger than statement coverage?

5. When is exploratory testing especially useful?

6. What does Given–When–Then usually represent?

Suggested answers:

1) black-box.

2) EP groups equivalent values; BVA targets ordered partition edges.

3) feasible decision rules.

4) branch coverage requires decision outcomes, not just executable statements.

5) weak/changing specs, time pressure, skilled testers, learning needed.

6) context, action and expected result.

INTERACTIVE Run as a group quiz: learners answer without notes, then justify their answer.



Questions? No?

Exercise timeeee..